

Lab. 05
Building a Web API with Express

Contents

1) Recap: From Workbench to Code to the Web
2) How the web works: requests, responses, URLs
3) HTTP methods and status codes
4) What a web framework is, and why Express (the counterfactual)
5) A first Express server; routes that return JSON
6) Middleware: the request pipeline behind app.use()
7) Serving data from MySQL through your API
8) Route parameters, query strings and the request body (POST)
9) A tiny web page that calls your API with fetch()
10) Where we're going next (Lab 6)
11) Summary

Recap: From Workbench to Code to the Web

- In Lab 3 you queried MySQL by Workbench; in Lab 4 a Node.js program ran those queries with mysql2.
- But a script only you can run isn't an application, other people reach software over the web.
- Today we put your database behind a web API: a program other computers can call to get and add data.
- Same SQL, same mysql2, now wrapped in a web server built with Express.

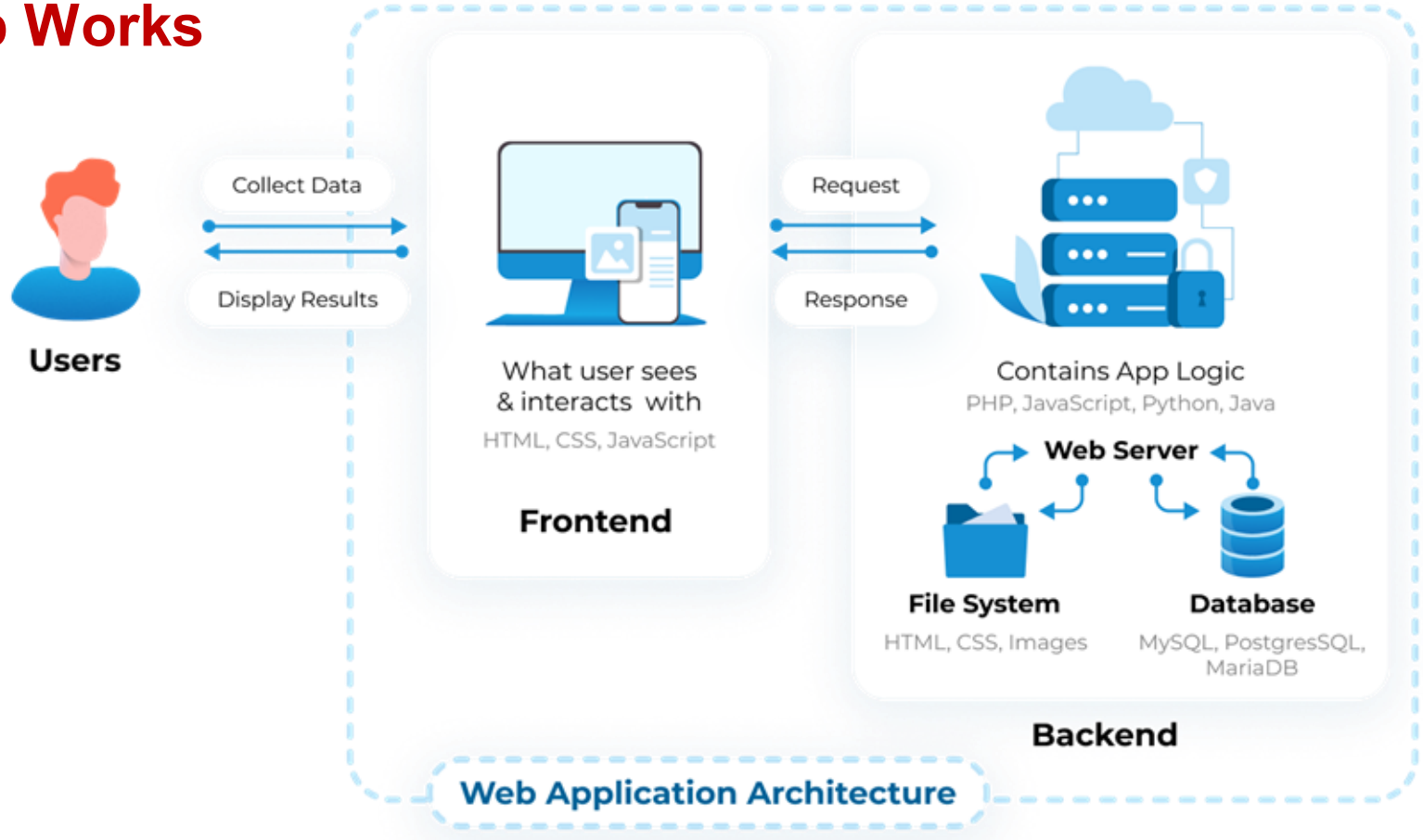
How the web works: requests, responses, URLs

How the Web Works

- The web is built on two roles: a client asks, a server answers.
- Your browser is a client; it sends a request to a server, which sends back a response.
- A request names a URL (where) and a method (what to do); the response carries data and a status.
- Our server's responses will be JSON, the same objects-and-arrays shape you saw in Lab 1 and Lab 4.

How the web works: requests, responses, URLs (Contd.)

How the Web Works



How the web works: requests, responses, URLs (Contd.)

Website vs. Web API: The Critical Distinction

Same transport (HTTP), same methods, same status codes: the difference is who's consuming the response, and what shape it is.

A Website

- Responds with HTML (plus CSS/JS) a document meant to be rendered
- Consumed by a human, through a browser
- Example: GET / returns a full page to look at
- The browser turns the response into pixels

A Web API

- Responds with data: usually JSON, no layout at all
- Consumed by a program: browser JS, a mobile app, another server, Postman
- Example: GET /students returns rows as JSON
- The caller decides what to do with the data

Today's server is a Web API. Slide 21 adds a tiny website that calls it, two separate things, talking over HTTP.

How the web works: requests, responses, URLs (Contd.)

HTTP: Methods and Status Codes

Every request uses a method that says what it wants; every response carries a status code.

```
GET     /students       -- read a list
GET     /students/7     -- read one item
POST    /students       -- create a new item
PUT     /students/7     -- update an item
DELETE  /students/7     -- remove an item

200 OK   201 Created   404 Not Found   500 Server Error
```

Method + URL describe the action; the status code reports how it went. These map neatly onto CRUD.

What a web framework is, and why Express (the counterfactual)

- Node's own http module can serve the web, but you'd hand-parse every URL, check every method with if-statements, and read raw request bytes yourself.
- A framework is reusable scaffolding: it handles that repetitive plumbing so you write only your own logic.
- Express is one such framework for Node.js: not the only one (Fastify, Koa and NestJS also exist), but it is the smallest, most widely used, and best documented.
- You describe routes (a method plus a path) and what each should do; Express does the parsing, matching and wiring.

What a web framework is, and why Express (the counterfactual) (Contd.)

The Counterfactual: Express vs. Plain Node

Both can serve the same route. One makes you build the plumbing yourself.

Same result, far less boilerplate, this is the concrete reason the course uses Express.

Plain Node http	Express
• Manually read req.url and split off the path • Manually check req.method with if / else • Manually collect and parse the request body as raw bytes • Manually set headers and call res.end() • Every route repeats this by hand	• app.get('/students/:id', handler), path and params parsed for you • One method call per HTTP verb: app.get, app.post, ... • express.json() middleware parses the body into req.body • res.json(data) sets headers and sends in one call • You write only the logic inside each handler

A first Express server; routes that return JSON

Install Express, create an app, define one route, and start listening on a port.

```
// npm install express
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello from CSS326!');
});

app.listen(3000, () => console.log('http://localhost:3000'));
```

```
// run this in command prompt
npm init -y
npm install express
// save this snippet as “app.js”
node app.js
// view it on browser
http://localhost:3000
```

app.get(path, handler) defines a route; app.listen(port) starts the server. Visit the URL in a browser.

A first Express server; routes that return JSON (Contd.)

Routes and Responses: A route handler receives the request (req) and replies with the response (res).

```
app.get('/hello', (req, res) => {
  res.send('plain text');
});

app.get('/data', (req, res) => {
  res.json({ course: 'CSS326', year: 3 });
});
```

```
// add to snippet “app.js”
node app.js
// view it on browser
http://localhost:3000/hello
http://localhost:3000/data
```

res.send() returns text; res.json() returns JSON ; exactly what an API should send.

Middleware: the request pipeline behind app.use()

The Request Pipeline

- A middleware function sits between the incoming request and your route handler, and can inspect or modify it along the way.
- app.use(fn) registers middleware that runs on every matching request, in the order you add it, order matters.
- express.json() is middleware: without it, req.body is undefined; with it, the JSON body is parsed into an object before your handler runs.
- express.static('public') is also middleware: it checks if a request matches a file in public/ before your routes even see it.
- You'll meet both again on the slides ahead, recognize them as the same pattern: a function that runs on the way in.

Middleware: the request pipeline behind app.use() (Contd.)

Lab Exercise 1: *Your first server*

- ❑ Create a project, run npm install express, and write app.js.
- ❑ Add a GET / route that responds with a welcome message.
- ❑ Start the server with app.listen(3000) and open http://localhost:3000.
- ❑ Add a second route, GET /data, that returns a small JSON object.

Starter Code:

```
const express = require('express');
const app = express();
app.get('/', (req, res) => res.send('____'));
app.listen(3000);
```

Serving data from MySQL through your API

Inside a route, run your Lab 4 query and send the rows back as JSON.

```
const express = require('express');
const mysql = require('mysql2/promise');
const app = express();

app.get('/students', async (req, res) => {
  const conn = await mysql.createConnection({ /* config */ });
  const [rows] = await conn.query('SELECT * FROM students');
  await conn.end();
  res.json(rows);
});

app.listen(3000, () => console.log('http://localhost:3000'));
```

```
// run this first
npm install mysql2
```

```
// For /* config */
host: 'localhost',
user: 'root',
password: root',
database: 'lab4'
```

Route parameters, query strings and the request body (POST)

Route Parameters: /students/:id

A colon marks a part of the path that varies; read it from req.params and pass it through a ?.

```
app.get('/students/:id', async (req, res) => {
  const [rows] = await conn.execute(
    'SELECT * FROM students WHERE id = ?',
    [req.params.id]);
  if (rows.length === 0) return res.status(404).json({ error: 'not found' });
  res.json(rows[0]);
});
```

req.params.id is the value from the URL. Still parameterized with ? the same safe habit as Lab 4. Try this "http://localhost:3000/students/1"

Route parameters, query strings and the request body (POST) (Contd.)

Lab Exercise 2: *Read from the database*

- Write GET /students that returns all students as JSON.
- Write GET /students/:id that returns one student, or 404 if the id is unknown.
- Use a ? placeholder for the id, just like Lab 4.
- Test both routes in your browser.

Starter code:

```
app.get('/students/:id', async (req, res) => {
  const [rows] = await conn.execute(
    'SELECT * FROM students WHERE id = ?', [req.params.id]);
});
```

Route parameters, query strings and the request body (POST) (Contd.)

Status Codes and Errors: Return the right status, and wrap each handler so a failure becomes a 500 instead of a crash.

```
app.get('/students', async (req, res) => {
  try {
    const [rows] = await conn.query('SELECT * FROM students');
    res.json(rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

200 by default, 201 on create, 404 when nothing matches, 500 when the server itself fails.

Route parameters, query strings and the request body (POST) (Contd.)

Query Strings and the Request Body

Filters arrive as ?key=value (req.query); data sent to create things arrives in req.body.

```
// GET /students?major=Data%20Science
app.get('/students', async (req, res) => {
  const major = req.query.major;
  // ... use major in a WHERE ... = ? query ...
});


app.use(express.json());   // lets Express read JSON request bodies
// now req.body holds the posted JSON object
```

express.json() is middleware: it parses the JSON body so req.body becomes a normal object.

Route parameters, query strings and the request body (POST) (Contd.)

Creating Data: POST

A POST request carries a new item in its body; insert it and reply 201 Created.

```
app.use(express.json());
 app.post('/students', async (req, res) => {
  const { name, major, year } = req.body;
  const [r] = await conn.execute(
    'INSERT INTO students (name, major, year) VALUES (?, ?, ?)',
    [name, major, year]);
  res.status(201).json({ id: r.insertId });
});
```

Destructure req.body, insert with ?, then return 201 and the new id. Test it with a REST client or fetch.

Route parameters, query strings and the request body (POST) (Contd.)

Lab Exercise 3: *Create with POST*

- ❑ Add app.use(express.json()) so Express can read JSON bodies.
- ❑ Write POST /students that inserts the posted student and returns 201 with the new id.
- ❑ Wrap your route handlers in try/catch and return 500 on error.
- ❑ Test the POST with a REST client (or the fetch page from the next slide).

Starter code:

```
app.post('/students', async (req, res) => {
  const { name, major, year } = req.body;
  // INSERT ... VALUES (?, ?, ?)
  res.status(201).json({ id: r.insertId });
});
```

Route parameters, query strings and the request body (POST) (Contd.)

Lab Exercise 3: *Create with POST (Continued)*

- Test the POST with a REST client (or the fetch page from the next slide).

Code:

```
curl -X POST http://localhost:3000/students ^
  -H "Content-Type: application/json" ^
  -d
"{\"name\":\"Daniel\",\"year\":3,\"major\":\"
Data Science\"}"
```

curl sends HTTP requests from the terminal. -X POST sets the method, -H adds a header, -d sets the body.

A tiny web page that calls your API with fetch()

Serve a static HTML page, then let the browser fetch your API and show the data (*Exercise 4: only for checking the output: app.use(express.static('public')); // in the app.js*).

```
 <ul id="list"></ul>
<script>
  async function load() {
    const rows = await (await fetch('/students')).json();
    document.getElementById('list').innerHTML =
      rows.map(s => `<li>${s.name} -
${s.major}</li>`).join('');
  }
  load();
</script>
```

express.static('public') serves the page; fetch() calls your API : two separate programs, joined by HTTP.

A tiny web page that calls your API with fetch() (Contd.)

Why "index.html"?: app.use(express.static('public')) doesn't hardcode a filename, it follows a much older web-server convention

```
// Any web server -- Apache, Nginx, Express -- follows the same rule:
// a request for a DIRECTORY, with no filename given,
// is answered with a file literally named index.html
// if one exists in that directory.

GET /          -->  looks for  public/index.html
GET /about     -->  looks for  public/about/index.html
GET /students.js -->  serves    public/students.js  (named directly)
```

It's a convention, not an Express feature: index.html is simply the default document a directory serves.

A tiny web page that calls your API with fetch() (Contd.)

Putting It Together

- Keep it tidy: db.js holds the connection config, app.js holds the server and routes.
- Each route does one thing: read the request, run a parameterized query, send a JSON response.
- A 'public' folder holds static pages the browser can load and that call your API.
- That is the whole shape of a small web application: browser → Express → MySQL → back again.

A tiny web page that calls your API with fetch() (Contd.)

Good Habits

- ❑ Send JSON with the right status code: 200, 201, 404, 500, don't make every response a 200.
- ❑ Keep parameterizing every value with ? inside your route handlers.
- ❑ Wrap handlers in try/catch so one bad request can't crash the whole server.
- ❑ Keep configuration (host, password) in one place, not copied into every route.

Where We're Going Next

- You now have a working web API: routes over HTTP that read and write your MySQL database.
- In Lab 6 we will organize the code into a clear structure and connect a fuller front-end page to the API.
- Everything you have built: schema, SQL, mysql2, Express is the foundation for your course project.

Summary

- The web is request/response between a client and a server; a Web API returns data (JSON) rather than a page.
- A framework such as Express removes the boilerplate of routing, parsing and responses that plain Node's http module leaves to you.
- Middleware (app.use) runs on the way in: express.json() and express.static() are both examples you used today.
- Run your mysql2 queries inside route handlers; read req.params, req.query and req.body; reply with 200/201/404/500.
- index.html is not an Express feature, it's the default document any web server serves for a directory request.
- Complete Exercises 1–4 in VS Code during class, against your Lab 3 database.

Thanks…